

Module 2: Text File Handling (Character-Based) - Complete Study Notes

Welcome to **Module 2** of our Java File Handling course! Building on Module 1's `File` class, this module dives into reading and writing **text-based files** using character streams. Text files (e.g., `.txt`, `.csv`, `.log`) store human-readable content, so we use `Reader` and `Writer` classes for efficient, encoding-aware handling (default UTF-8).

By the end of this module, you'll:

- Write and read text files using basic and buffered streams.
- Append to existing files without overwriting.
- Build reusable methods for common text operations.
- Handle errors like file-not-found gracefully.

Key Concepts:

- **Character Streams vs. Byte Streams:** Character streams (`Reader` / `Writer`) handle Unicode text automatically; byte streams (Module 3) are for raw binary.
- **Buffering:** Wrap streams in `BufferedReader` / `BufferedWriter` for performance—reduces direct disk access.
- **Try-With-Resources:** Always use this for auto-closing streams to avoid leaks.
- **IOException:** Wrap all I/O in try-catch.

Prerequisites: Complete Module 1. We're using `java.io` package.

To visualize the stream hierarchy, here's a helpful diagram:

1. Core Concepts: Character Streams Overview

Character streams abstract text I/O as a sequence of `char` (16-bit Unicode). Key classes:

- **Abstract Base:** `Reader` (input), `Writer` (output).
- **File-Specific:** `FileReader` / `FileWriter` —simple, tied to files.
- **Buffered:** `BufferedReader` / `BufferedWriter` —add efficiency for line-based ops.
- **Why Character?** Handles accents/emojis; bytes would require manual decoding.

Import Statements (add at top):

```
import java.io.*; // Covers File, IOException, Reader, Writer, etc.
```

2. Step-by-Step Code Examples with Comments

Examples build progressively. Each is a runnable class/method. Create a "data" folder from Module 1 for testing.

Step 1: Writing to a Text File

Start with `FileWriter` for basics, then buffer for speed.

Sample Code: `TextWriterBasic.java`

```
public class TextWriterBasic {
    public static void main(String[] args) {
        // Target file—use File from Module 1 if needed
        String filename = "data/output.txt";

        // Use try-with-resources: Auto-closes writer even if exception occurs
        try (FileWriter writer = new FileWriter(filename)) { // Creates file if missing
            // Write a string—converts to chars and flushes to disk
            writer.write("Hello, Java Text Handling!\n"); // \n for newline

            // Write another line
            writer.write("This is line 2.");

            System.out.println("Text written successfully!");
        } catch (IOException e) {
            // Catch common errors: File not writable, disk full, etc.
            System.err.println("Write error: " + e.getMessage());
            e.printStackTrace(); // Debug: Shows full stack
        }
    }
}
```

Output: Creates "output.txt" with two lines. Check with a text editor.

Buffered Version (Better for multiple writes):

Sample Code: `TextWriterBuffered.java`

```
public class TextWriterBuffered {
    public static void main(String[] args) {
        String filename = "data/output_buffered.txt";

        try (BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(filename))) {
            // Write with buffer: Collects data in memory, writes in chunks
            bufferedWriter.write("Buffered writing is efficient!\n");

            // newLine() adds OS-specific line ending (\n on Unix, \r\n on Windows)
            bufferedWriter.newLine();
            bufferedWriter.write("Line 2: See the difference.");

            // Optional: flush() forces immediate write (auto on close)
            bufferedWriter.flush();

            System.out.println("Buffered text written!");
        } catch (IOException e) {
            System.err.println("Buffered write error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Why Buffer? Fewer disk hits—ideal for loops.

Step 2: Reading from a Text File

`FileReader` for chars; `BufferedReader` for lines.

Sample Code: `TextReaderBasic.java` (Assumes "output.txt" from Step 1 exists)

```
public class TextReaderBasic {  
    public static void main(String[] args) {  
        String filename = "data/output.txt";  
  
        try (FileReader reader = new FileReader(filename)) {  
            // Read char by char—inefficient for large files  
            int charValue; // read() returns int (Unicode value) or -1 for EOF  
            StringBuilder content = new StringBuilder(); // Build string from chars  
  
            while ((charValue = reader.read()) != -1) { // Loop until end of file  
                content.append((char) charValue); // Cast int to char  
            }  
  
            System.out.println("Full content:\n" + content.toString());  
        } catch (IOException e) {  
            System.err.println("Read error: " + e.getMessage());  
            e.printStackTrace();  
        }  
    }  
}
```

Output:

```
Full content:  
Hello, Java Text Handling!  
This is line 2.
```

Buffered Version (Line-by-line—recommended):

Sample Code: TextReaderBuffered.java

```
public class TextReaderBuffered {  
    public static void main(String[] args) {  
        String filename = "data/output.txt";  
  
        try (BufferedReader bufferedReader = new BufferedReader(new FileReader(filename))) {  
            String line; // Each readLine() returns a String or null for EOF  
  
            System.out.println("Reading line by line:");  
            int lineNumber = 1;  
            while ((line = bufferedReader.readLine()) != null) { // Loop until null  
                System.out.println(lineNumber + ": " + line); // Process each line  
                lineNumber++;  
            }  
        } catch (IOException e) {  
            System.err.println("Buffered read error: " + e.getMessage());  
            e.printStackTrace();  
        }  
    }  
}
```

Output:

```
Reading line by line:  
1: Hello, Java Text Handling!  
2: This is line 2.
```

Step 3: Append Mode

Add to existing files without overwriting.

Sample Code: TextAppender.java (Appends to "output.txt")

```
public class TextAppender {  
    public static void main(String[] args) {  
        String filename = "data/output.txt";  
  
        try (BufferedWriter appender = new BufferedWriter(new FileWriter(filename, true))) { // 1  
            // Append new content—file grows instead of resetting  
            appender.newLine(); // Add blank line for readability  
            appender.write("Appended line: Welcome to Module 2!");  
  
            System.out.println("Appended successfully!");  
        } catch (IOException e) {  
            System.err.println("Append error: " + e.getMessage());  
            e.printStackTrace();  
        }  
    }  
}
```



Result: "output.txt" now has original + new line.

3. Practice: Building Reusable Methods

Modularize for reuse—like a text utility class.

Classwork 1: Simple Text Processor

Methods for write/read entire files.

Complete Class: TextProcessor.java

```
import java.io.*;

public class TextProcessor {
    /**
     * Writes a list of strings to a file, one per line.
     * @param filename Output file path
     * @param lines Array of strings to write
     */
    public static void writelines(String filename, String[] lines) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            for (String line : lines) { // Loop through array
                writer.write(line);
                writer.newLine(); // End each line properly
            }
            System.out.println("Lines written to " + filename);
        } catch (IOException e) {
            System.err.println("Write lines error: " + e.getMessage());
        }
    }

    /**
     * Reads all lines from file into a List<String>.
     * @param filename Input file path
     * @return List of lines, or empty if error
     */
    public static java.util.List<String> readLines(String filename) {
        java.util.List<String> lines = new java.util.ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            while ((line = reader.readLine()) != null) {
                lines.add(line); // Collect in list
            }
            System.out.println("Read " + lines.size() + " lines from " + filename);
        } catch (IOException e) {
            System.err.println("Read lines error: " + e.getMessage());
        }
        return lines;
    }

    public static void main(String[] args) {
        // Test: Write sample lines
        String[] sample = {"Line 1: Java", "Line 2: File Handling", "Line 3: Rocks!"};
        writelines("data/sample.txt", sample);
    }
}
```

```
// Read back
java.util.List<String> read = readLines("data/sample.txt");
read.forEach(System.out::println); // Print each (Java 8+ lambda)
}
}
```

Sample Output:

```
Lines written to data/sample.txt
Read 3 lines from data/sample.txt
Line 1: Java
Line 2: File Handling
Line 3: Rocks!
```

Depth Adder: Simple Logger (Append with Timestamp)

For logging—appends dated messages.

Enhanced Class: TextLogger.java


```

import java.io.*;
import java.time.LocalDateTime; // Java 8+ for timestamps
import java.time.format.DateTimeFormatter;

public class TextLogger {
    private static final String LOG_FILE = "data/app.log"; // Constant for file

    /**
     * Appends a message with timestamp to log.
     * @param message The log message
     */
    public static void log(String message) {
        String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
        // Format: "[timestamp] message"
        String logEntry = "[" + timestamp + "] " + message;

        try (BufferedWriter logger = new BufferedWriter(new FileWriter(LOG_FILE, true))) {
            logger.write(logEntry);
            logger.newLine();
            System.out.println("Logged: " + logEntry);
        } catch (IOException e) {
            System.err.println("Log error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        log("Application started.");
        log("Processing text files...");
        log("Module 2 complete!");
    }
}

```

Result: "app.log" with timestamped entries. **Why?** Real-world use for debugging.

4. Tasks and Exercises

Practice time! Build on examples.

Task 1: Basic Writer/Reader (Beginner - 10 mins)

- Class: `EchoWriter` .
- Method: `echoToFile(String input, String output)` —read from `input.txt` , write reversed lines to `output.txt` (e.g., reverse each line's words with `String.split`).
- Test: Create "input.txt" with 3 lines, run, verify output.
- **Expected:** Handle empty file (write nothing).

Task 2: Appending Diary (Intermediate - 15 mins)

- Class: `DiaryAppender` .
- Method: `addEntry(String filename, String entry)` —append with date (use `LocalDateTime`).
- Test: Append 2 entries to "diary.txt", read back with `readLines` from Classwork 1.
- **Challenge:** If file > 1000 chars, truncate old entries.

Task 3: Line Counter/Processor (Advanced - 20 mins)

- Extend `TextProcessor` : Add `countWords(String filename)` —read lines, count total words (split by space).
- Bonus: Filter lines > 20 chars, write to "filtered.txt".
- **Why?** Teaches processing pipelines.

Task 4: Robust Reader (Debugging - 10 mins)

- Modify `readLines` to handle `FileNotFoundException` separately: Print "Create file first!" and return empty list.
- Test: Call on non-existent file—ensure no crash.

Grading Rubric (Self-Assess):

- Code compiles/runs: 40%
- Comments on I/O blocks: 30%
- Uses buffering/try-with: 20%
- Clean, readable output: 10%

5. Common Pitfalls and Best Practices

Pitfall	Fix
Overwriting files accidentally	Use <code>true</code> in <code>FileWriter(filename, true)</code> for append
Encoding issues (e.g., garbled UTF-8)	Specify charset: <code>new FileWriter(filename, Charset.forName("UTF-8"))</code>
Not closing streams	Always <code>try-with-resources</code> —no manual <code>close()</code> needed
Reading huge files into memory	Use <code>readLine()</code> loop—process one at a time
Forgetting <code>\n</code>	Use <code>newLine()</code> for cross-platform lines

Pro Tip: For CSV, use `BufferedReader + line.split(",")` ; libraries like `OpenCSV` for complex cases (but stick to basics here).

Resources:

- Oracle Docs: [BufferedReader](#)
- Practice: Write a word counter for a book excerpt.

Next Up: Module 3 (Binary Handling). Tackle tasks—share code for feedback!